SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: **Artificial Intelligence** **COURSE CODE:CSA1708**

COURSE OUTCOME COVERED

CO1:ApplyAIbasics suchasintelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3

Duration : 1Hour
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Ganesh Ram V (1924260258) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1: Doctor Shift Scheduling (Backtracking Search)

1. CSP formulation

- **Variables:**D1, D2, D3 (the shift of each doctor).
- **Domain:** {Morning (M), Afternoon (A), Night (N)}, ordered M < A < N.
- **Constraints:** C1: D1 ≠ N. C2: D2's shift is earlier than D3's shift. C3: D3 ≠ M. C4/C5: all three shifts are different (one doctor per shift, each doctor exactly one shift).
- **Unary pruning first:** C1 gives D1 ∈{M, A}; C3 gives D3 ∈{A, N}; D2 ∈{M, A, N}.
- **Search settings:** variable order D1, D2, D3 and value order M, A, N. After every assignment, all constraints whose variables are already assigned are checked.

2. Backtracking trace

Step	Assignment tried	Constraint check	Result
1	D1 = M	C1 (D1 ≠ N): OK	Ac ce pt
2	D2 = M	All-different: M already used by D1 ✗	Reject
3	D2 = A	All-different OK. C2 cannot be checked yet (D3 unassigned)	Ac ce pt
4	D3 = M	C3 (D3 ≠ M) ✗, and M already used ✗	Reject
5	D3 = A	All-different: A already used by D2 ✗	Reject
6	D3 = N	All-different OK; C3 OK; C2: D2 = A < D3 = N ✓	Accept:complete assignment

Solution found (first one reached by the search): D1 = Morning, D2 = Afternoon, D3 = Ni ght .

3. Verification against every constraint

Constraint	Check
D1 cannot work Night	D1 works Morning✓
D2 must work before D3	Afternoon < Night✓
D3 cannot work Morning	D3 works Night✓
Only one doctor per shift	M, A, N each used once ✓
Every doctor has exactly one shift	D1, D2, D3 each assigned once ✓

4. Backtracking actually occurring (exploring the rest of the tree)

The search abovereachedasolution withouthavingtoundo an assignment, because the first branch was already consistent. Continuing the search to see whether the answer is unique shows real backtracking:

Step	Assignment tried	Constraint check	Result
7	Backtrack to D2; try D2 = N (D1 = M)	All-different OK	Ac ce pt

Step	Assignment tried	Constraint check	Result
8	D3 = M / A / N	M:C3 ✗ ;A:C2failssinceN is not earlier than A ✗ ; N: already used ✗	Allrejected:dead end, BACKTRACK
9	Backtrack to D1; try D1 = A	C1 OK	Ac ce pt
10	D2 = M	All-different OK	Ac ce pt
11	D3 = M, A	M: C3 ✗ and used; A: used by D1 ✗ All-different OK, C3 OK, C2: M < N ✓	Reject
12	D3 = N	C1 ✗(and N is not in D1's pruned domain)	Accept: second solution
13	D1 = N		Reject; search ends

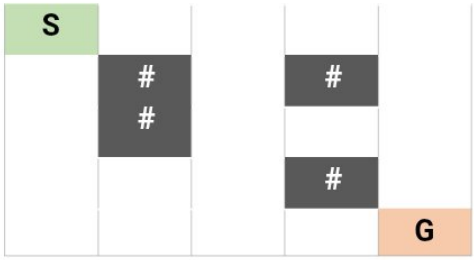
Final valid schedule: Morning = D1, Afternoon = D2, Night = D3.

Note: the constraints also allow a second valid schedule (Morning = D2, Afternoon = D1, Night = D3). The question asks for a valid assignment, so the first one found by the specified ordering is reported.

Question 2: Grid Robot Path Planning (5×5)

Assumption (please note): the question does not include the grid diagram, so the following standard 5×5 layout is assumed. Coordinates are (row, column), starting at (0,0) in the top-

left.S = (0,0), G=(4,4),obstacles(#)at(1,1),(1,3),(2,1)and(3,3).Thesamemethod applies to any other layout.



1. Problem formulation

- **State:** the robot's cell (r, c). **Initial state:** (0,0). **Goal test:** state = (4,4).
- **Actions:** Up, Down, Left, Right (blocked by obstacles and grid edges). **Step cost:** 1 per move.
- **Neighbour expansion order:** Up, Down, Left, Right. Ties are broken by first-in-first-out.
- **Manhattan heuristic:** h(n) = |r − 4| + |c − 4|. It never overestimates when moves cost 1, so itis admissible. h(S) =8.

A note on the question: BFS uses a plain FIFO queue and does not use a heuristic. A priority queue orderedby f(n) = g(n) + h(n) withtheManhattanheuristicis A* search. To cover every part of the question, BFS is traced first (FIFO queue), then the priority-queue version (A*) is traced, and the two results are compared.

2. BFS: node expansion and queue (frontier) updates

Visited nodes are marked when they are generated, so no cell enters the queue twice. The goal test is applied when a node is removed from the queue.

Step	Node expanded	New nodes added	Queue after the step (front → back)
0	-	-	(0 ,0)

Step	Node expanded	New nodes added	Queue after the step (front → back)
1	(0 ,0)	(1,0), (0,1)	(1,0), (0,1)
2	(1 ,0	(2 ,0)	(0,1), (2,0)
3	) (0	(0 ,2)	(2,0), (0,2)
4	,1)	(3 ,0)	(0,2), (3,0)
5	(2 ,0	(1,2), (0,3)	(3,0), (1,2), (0,3)
6	) (0	(4,0), (3,1)	(1,2), (0,3), (4,0), (3,1)
7	,2)	(2 ,2)	(0,3), (4,0), (3,1), (2,2)
8	(3 ,0	(0 ,4)	(4,0), (3,1), (2,2), (0,4)
9	) (1	(4 ,1)	(3,1), (2,2), (0,4), (4,1)
10	,2)	(3 ,2)	(2,2), (0,4), (4,1), (3,2)
11	(0 ,3	(2 ,3)	(0,4), (4,1), (3,2), (2,3)
12	) (4	(1 ,4)	(4,1), (3,2), (2,3), (1,4)
13	,0)	(4 ,2)	(3,2), (2,3), (1,4), (4,2)
14	(3 ,1	none	(2,3), (1,4), (4,2)
15	) (2	(2 ,4)	(1,4), (4,2), (2,4)
16	,2)	none	(4,2), (2,4)
17	(0 ,4	(4 ,3)	(2,4), (4,3)
18	) (4	(3 ,4)	(4,3), (3,4)
19	,1)	(4 ,4)	(3,4), (4,4)
20	(3 ,2	none	(4 ,4)
21	) (2	GOAL reached	
	,3)		
	(1 ,4		
	) (4		
	,2)		
	(2 ,4		-
	) (4		
	,3)		
	(3 ,4		
	) (4		
	,4)		

BFS result: path (0,0) →(1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4); cost = 8; nodes expanded = 21.

3. Priority-queue version with the Manhattan heuristic (A*)

Priority =f(n) = g(n)+ h(n).Ties arebrokenbysmallerh,thenbytheorder of insertion. Each queue entry is shown as node[f].

Step	Node expanded (g, h, f)	Children generated (g, h, f)	Priorityqueue after the step (best first)
0	-	-	(0 ,0) [f=8]
1	(0,0) g=0,h=8,f=8	(1,0)(g=1,h=7,f=8); (0 ,1)(g=1, h=7, f= 8) (2 ,0)(g=2, h=6,	(1,0)[f=8], (0,1)[f=8]
2	(1,0) g=1,h=7,f=8	f= 8) (3 ,0)(g=3, h=5, f= 8)	(2,0)[f=8], (0,1)[f=8]
3	(2,0) g=2,h=6,f=8		(3,0)[f=8], (0,1)[f=8]
4	(3,0) g=3,h=5,f=8	(4,0)(g=4,h=4,f=8); (3 ,1)(g=4, h=4, f= 8)	(4,0)[f=8], (3,1)[f=8], (0,1)[f=8]
5	(4,0) g=4,h=4,f=8	(4 ,1)(g=5, h=3, f= 8) (4 ,2)(g=6, h=2, f= 8) (3,2) (g=7,h=3,f=10); (4 ,3)(g=7, h=1, f= 8)	(4,1)[f=8], (3,1)[f=8], (0,1)[f=8]
6	(4,1) g=5,h=3,f=8		(4,2)[f=8], (3,1)[f=8], (0,1)[f=8]
7	(4,2) g=6,h=2,f=8		(4,3)[f=8], (3,1)[f=8], (0,1)[f=8], (3 ,2) [f=10]

Step	Node expanded (g, h, f)	Children generated (g, h, f)	Priority queue after the step (best first)
8	(4,3) g=7,h=1,f=8	(4 ,4)(g=8, h=0, f= 8)	(4,4)[f=8], (3,1)[f=8], (0,1)[f=8], (3 ,2) [f=10]
9	(4,4) g=8,h=0,f=8	GOAL reached	-

A* result: path (0,0) →(1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4); cost = 8; nodes expanded = 9.

4. Optimal path and cost

Optimal path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4),that is Down ×4, then Right ×4.

Optimal cost = 8. ThisisoptimalbecausetheManhattandistancefromSto Gis8,which is a lower bound on any path, and the path found has exactly that length. BFS is optimal here because every step costs 1. Using the heuristic, the priority-queue search reached the same answer after 9 expansions instead of 21.

Question 3: Autonomous Rescue Robot (Online Search)

a) Analysis of the constraints and their effect on decision-making

Constraint	What it implies	Effect on the robot's decisions
Moves: up, down, left, right	Discrete grid with 4 actions per state	Small branching factor (at most 4). A graph-search formulation with a visited set is enough.
Blocked cells (obstacles)	Someactions are illegal in some states	Obstacles must be excluded from the successor function. Newly discovered obstacles force the plan to be recomputed.
Limitedsensing (adjacent cells only)	Partially o bserv ab le envir onment; the map is not known in advance	The robot cannot plan the whole route offline. It must act, sense, update its map, and re-plan (online search).
Step cost 1, risky zone +2	Non-uniform costs: a normal move costs 1, entering a risky cell costs 3	Plain BFS is not optimal. A cost-sensitive search such as Uniform Cost Search or A* is needed, and a short route through danger may lose to a longer safe route.
Minimise total cost with safe navigation	Optimisation objective with a safety preference	The performance measure is the path cost. Risk is built into the cost, and ties are broken in favour of the safer route.
Dynamic knowledge update	The knowledge base changes during execution	The agent keeps an internal model (known free, blocked and risky cells) and revises its plan after every percept.

Priority of constraints: (1) never enter a blocked cell (hard constraint); (2) work only from whathas beensensed(information constraint);(3) minimise cost including the risky-zone penalty (optimisation); (4) tie-break towards safer cells.

b) Solution approach: online Uniform Cost Search with re-planning

The approach combines Uniform Cost Search (UCS) with an online agent loop. It is the same idea used by D* Lite style planners, simplified here. The cost of entering a cell is c(cell) = 1 for a normal cell and 3 for a risky cell.

- Step 1, initialise: the map memory marks S as known and free; all other cells are 'unknown'. The goal position G is known (the survivor's location).
- Step 2, sense: read the percepts for the four adjacent cells (free, blocked or risky) and update the map memory.
- Step 3, plan: run UCS from the current cell to G over the known map. Unknown cells are assumedtraversable at cost1(optimistic assumption)sothatthe robotisdrawn to explore towards G. UCS expands nodes in order of the lowest path cost g(n) using a priority queue.
- Step 4, act: execute only the first move of the cheapest plan, then move to that cell and add its costto the total.
- Step 5, re-plan: if a new obstacle or risky cell was sensed on the planned route, or the plan is no longer the cheapest, discard it and repeat from Step 2. Otherwise continue the existing plan.
- Step 6, terminate: stopwhenthe current cell equals G(survivor reached).If UCSfinds no route at all on the known map with unknown cells treated as free, the goal is unreachable and the agent reports failure.

Small illustration of the cost model: a route of 6 moves that crosses one risky cell costs 5 ×1+3 = 8; a safedetour of 8 moves costs 8.The costs tie,so the agent chooses the safe detour. If the detour needed 9 moves, the agent would accept the risky route (cost 8 < 9), because the problem asks to minimise total cost.

c) Application of AI concepts

Intelligent agent

Therobotisagoal-based and utility-based agent that maintains an internal model, so it is a model-based agent.

Component	In this problem
Performance measure	Reach the survivor at G with minimum total cost (moves plus risk penalty), without entering blocked cells
Envir onme nt	Grid of rooms with obstacles and risky zones
Actuators	Move up, move down, move left, move right
Sensors	Adjacent-cell sensing (free / blocked / risky); knowledge of own position

Agent function: the percept sequence (adjacent-cell readings so far) is mapped to an action (thefirst step of the current cheapest plan), through the internal map that is updated at every step.

Rationality

A rational agent selects the action expected to maximise its performance measure, given the percept sequence sofar anditsbuilt-inknowledge. Here this means therobot always chooses themove that liesonthecheapestknown path toG, andit re-plans assoon asnew percepts change the costs. Rationality is not omniscience: the robot cannot avoid an obstacle it has not yet sensed, but it behaves optimally with the information it has. Exploring towards G, rather than moving at random, is the rational way of dealing with the unknown parts of the map.

Environment type

Property	Classification	R ea son
Observability	Partially observable	Only adjacent cells are sensed
Agents	Single agent	Only the robot acts (survivors and hazards are treated as static)
De ter minism	De ter minis tic	A move action leads to the intended cell; costs are fixed
Episodic / sequential	S equ ent ial	Each move affects later options and the total cost
Static / dynamic	Static (mostly)	The layout does not change while the robot decides; if debris or fire can change it, the environment is dynamic and the same re-planning loop still applies
Discrete / continuous	Dis c re te	Grid cells and four discrete actions
Known / unknown	Unknown at the start	The robot has to learn the map while acting (online search)

d) Justification of the chosen approach

- **Why not BFS:** BFS minimises the number of steps, not the cost. With the +2 risky-zone penalty it can return a route that is cheap in moves but expensive in cost.
- **Why not offline search (plan once, then execute):** offline UCS or A* needs the full map inadvance.With adjacent-only sensing theplanwouldbreakas soonas an unexpected obstacle appears.
- **Why UCS:**it iscompleteandoptimal forany non-negativestepcosts, handles the different costs of normal and risky cells correctly, and needs no heuristic. A* with an admissible heuristic such as Manhattan distance (which stays admissible because the minimumcost ofany move is 1) is a validspeed-up to the sameplanner.
- **Why the online loop:** it is the only structure that fits partial observability: sense, update knowledge, plan, act, repeat. It keeps the robot safe because blocked cells are never entered and risk is priced into every plan.
- **Trade-offs:**re-planning repeats some computation,and the totalcost foundonline may be higher than the cost of the best path computed with a fully known map. This is unavoidable when the environment is partially observable, and the approach still makes the best decision possible at every step.